

AI Chat Assist - Complete Technical Documentation

A professional implementation-focused document for a modern AI chat application built with Next.js, React, TypeScript, Groq, and Google Gemini.

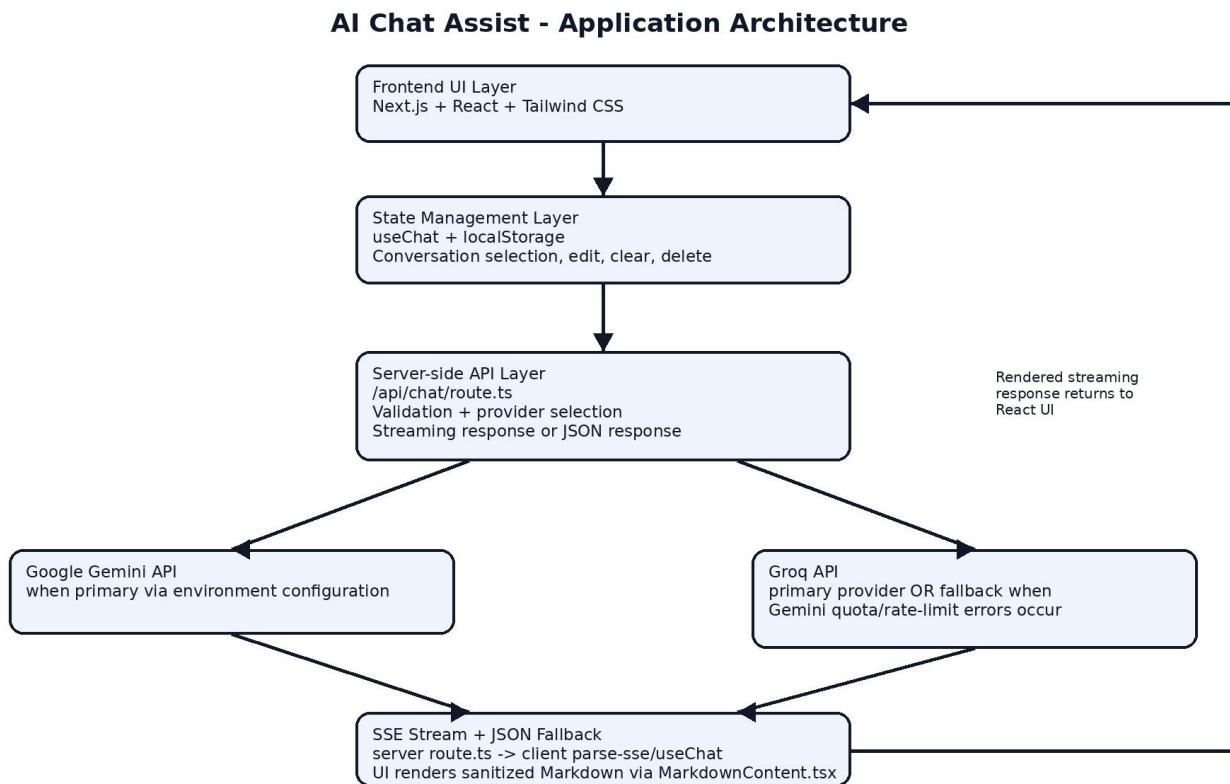
1. Project Overview

AI Chat Assist is a modern AI-powered conversational web application that enables users to interact with AI models using streaming responses similar to mainstream Generative AI platforms.

The project is structured with production-style frontend engineering practices: reusable components, secure server-side API communication, type-safe development, responsive design, persistent conversation history, and graceful error handling.

2. Architecture Diagram

The diagram below shows the end-to-end flow from the user interface to state management, secure API handling, provider selection, streaming response processing, JSON fallback, and final UI rendering.



The application routes all AI calls through Next.js API routes (/api/chat) so API keys never reach the browser. Per request, the server selects one primary provider. If Gemini is primary and returns quota or rate-limit errors, the same request is retried with Groq when GROQ_API_KEY is configured. When AI_PROVIDER=groq or only GROQ_API_KEY is configured, Groq handles the request directly with no Gemini fallback.

Flow:

/api/chat (provider.ts → gemini OR groq)

→ SSE stream (default)

→ client parseSseStream / useChat

→ if no deltas or streaming fails → client JSON retry (stream:false)

- MarkdownContent rendering + sanitization
- UI update

3. Key Features

- Real-time AI chat with streaming responses
- Multi-conversation sidebar with new, switch, and delete conversation actions
- Persistent conversation history using localStorage
- Dynamic message rendering without page refresh
- Loading states, streaming status updates, and user-friendly error handling
- Groq and Google Gemini AI integrations with provider resolution rules
- Groq fallback when Gemini quota or rate-limit errors occur
- Secure API communication through Next.js server-side API routes
- Markdown rendering with GFM support and sanitization
- Copy message and copy code block functionality
- Edit and resend user prompts
- Clear active chat and delete entire conversation flows
- Input validation for empty messages and 4000-character limit
- Responsive mobile and desktop layouts
- Unit testing with Vitest and React Testing Library

4. Technology Stack

Technology	Purpose
Next.js 16	Full-stack framework for UI and API routes
React 19	UI component rendering
TypeScript	Static type safety and maintainable contracts
Tailwind CSS 4	Responsive styling and custom UI design
Groq API	AI provider integration
Google Gemini API	AI provider integration
react-markdown	Markdown rendering
remark-gfm + rehype-sanitize	GitHub-flavored Markdown support and output sanitization
localStorage	Client-side conversation persistence
Vitest + React Testing Library	Unit testing for utilities and components

5. Feature Implementation and Design Decisions

This section maps the main assessment requirements and implemented functionality to the actual project files, core logic, and execution flow. The goal is to show not only what was built, but also how the implementation is organized and why it is maintainable.

Feature / Requirement	Implemented In	Core Logic and Flow
Responsive Chat UI	src/components/ChatApp.tsx, ChatHeader.tsx, ChatInput.tsx, MessageList.tsx, globals.css	The UI is composed from focused React components for the sidebar, header, message list, input composer, and error banner. Tailwind CSS classes provide a responsive desktop layout and mobile sidebar overlay, keeping presentation concerns separate from chat logic.
AI API Integration	src/app/api/chat/route.ts, src/lib/ai/provider.ts, src/lib/ai/gemini.ts, src/lib/ai/groq.ts	All requests go through the Next.js server route so provider keys remain server-only. provider.ts resolves the active provider from AI_PROVIDER and environment variables, then routes the request to Gemini or Groq based on configuration.
Provider Fallback Logic	src/lib/ai/provider.ts	When Gemini is the selected provider, quota or rate-limit errors are detected and the request is retried with Groq if GROQ_API_KEY exists. When Groq is explicitly selected, the app uses Groq directly and does not call Gemini.

Streaming AI Responses	src/app/api/chat/route.ts, src/lib/ai/stream-events.ts, src/lib/stream/parse-sse.ts, src/hooks/useChat.ts	The API route returns text/event-stream responses and sends status, delta, error, and done events. On the client, useChat reads the stream through parseSseStream and progressively updates the assistant message as chunks arrive.
JSON Fallback	src/hooks/useChat.ts, src/app/api/chat/route.ts	If streaming does not produce content or the streaming request fails, the client retries the same prompt using non-streaming JSON mode. This improves reliability across browser/network conditions while preserving the same server-side validation and provider logic.
Dynamic Message Rendering	src/hooks/useChat.ts, src/components/MessageList.tsx, src/components/MessageBubble.tsx	Messages are stored in React state and updated immediately after a user sends a prompt. Assistant responses are appended or updated in-place as stream deltas arrive, so the UI refreshes without a page reload.
State Management	src/hooks/useChat.ts	The custom useChat hook centralizes conversations, active conversation selection, loading state, streaming phase, errors, and user actions. Keeping this logic in one hook makes the UI components simpler and easier to test or extend.
Conversation Persistence	src/lib/storage.ts, src/lib/conversations.ts, src/hooks/useChat.ts	Chat history is saved in localStorage under a dedicated conversation store. The storage layer normalizes data, sorts conversations by updated time, and also includes a legacy message migration path.
Multi-conversation Sidebar	src/components/ChatSidebar.tsx, src/hooks/useChat.ts, src/lib/conversations.ts	The sidebar lists conversations with title, preview, relative time, active-state styling, and delete action. useChat handles new conversation creation, selection, deletion, and fallback to a fresh conversation when the last thread is removed.
Clear Chat vs Delete Conversation	src/components/ChatHeader.tsx, src/components/ChatSidebar.tsx, src/hooks/useChat.ts	Clear chat is a header action that removes messages only from the active conversation and resets its title to New chat. Delete conversation is a sidebar action that removes the entire thread from the stored conversation list.
Input Validation	src/lib/validation.ts, src/hooks/useChat.ts, src/app/api/chat/route.ts, src/components/ChatInput.tsx	The app rejects empty messages and messages over 4000 characters. Validation is applied on both client and server, while the input component also disables submission for blank messages and loading state.
Loading and Error Handling	src/hooks/useChat.ts, src/components/ErrorBanner.tsx, src/components/LoadingIndicator.tsx, src/components/ChatSkeleton.tsx	The hook tracks isLoading and streamPhase values such as thinking and streaming. Errors from validation, network failures, provider failures, or server responses are shown through a dismissible ErrorBanner instead of silently failing.
Markdown Rendering and Sanitization	src/components/MarkdownContent.tsx, src/lib/markdown.ts, src/components/CodeBlock.tsx	Assistant responses are rendered with react-markdown, remark-gfm, and rehype-sanitize for safer rich formatting. Code blocks are delegated to a CodeBlock component, improving readability and enabling copy-friendly output.
Copy Actions	src/components/CopyButton.tsx, src/components/MessageActions.tsx, src/components/CodeBlock.tsx, src/lib/clipboard.ts	Reusable copy behavior is separated into a clipboard utility and UI buttons. Users can copy assistant messages and code blocks, which improves practical usability for technical responses.
Edit and Resend Prompt	src/hooks/useChat.ts, src/components/MessageActions.tsx, src/components/MessageBubble.tsx	When a user edits a previous prompt, the hook trims the conversation after that message and regenerates the assistant reply using only the prior history. If the edited message is the first user prompt, the conversation title is refreshed as well.
Secure Environment Configuration	.env.example, src/lib/ai/provider.ts, src/app/api/chat/route.ts	The README and environment file document AI_PROVIDER, API keys, and model names. Keys are read only in server-side code, preventing exposure in browser bundles or client components.
Testing and Validation	src/**/*.test.ts, src/test/setup.ts, vitest.config.ts	Vitest and React Testing Library are used for utility and component tests, including validation, storage, conversation handling, formatting, parse-sse, and ChatInput behavior. This gives reviewers confidence that important flows are not just manually tested.
Deployment Readiness	next.config.ts, package.json, README.md	The app is structured as a Next.js project that can run locally and deploy cleanly to Vercel. The documented commands cover development, production build, linting, and testing for reviewer verification.

6. Application Architecture Principles

- Reusable UI component architecture separates visual rendering from business logic.
- The dedicated useChat hook manages conversation state, API calls, streaming updates, edits, deletion, clearing, and errors.
- Server-side API routes protect environment variables and centralize provider communication.
- SSE streaming is used for responsive user experience, with JSON fallback for reliability.
- TypeScript interfaces in src/lib/types.ts keep message, conversation, request, and response shapes consistent across the app.
- localStorage persistence keeps the implementation lightweight while still supporting multi-conversation history.

7. Project Structure

```
AI-Chat-Assist/  
├── src/  
│   ├── app/  
│   │   ├── api/chat/route.ts  
│   │   ├── globals.css  
│   │   ├── layout.tsx  
│   │   └── page.tsx  
│   ├── components/  
│   ├── hooks/  
│   ├── lib/  
│   │   ├── ai/           (groq, gemini, provider, stream-events)  
│   │   ├── stream/       (parse-sse)  
│   │   ├── types.ts  
│   │   ├── conversations.ts  
│   │   ├── storage.ts  
│   │   └── ...  
│   └── test/setup.ts  
├── public/  
├── vitest.config.ts  
├── next.config.ts  
├── .env.example  
├── package.json  
└── README.md
```

The folder structure separates app routes, reusable components, hooks, AI provider integrations, stream parsing, utilities, type definitions, and test setup. This organization makes the project easier to review, maintain, and extend.

8. Environment Configuration

AI_PROVIDER=groq

GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=llama-3.3-70b-versatile

GEMINI_API_KEY=your_gemini_api_key
GEMINI_MODEL=gemini-2.5-flash

Configuration	Primary Provider
AI_PROVIDER=groq	Groq
AI_PROVIDER=gemini	Gemini
Both keys set, AI_PROVIDER unset	Gemini (default)
Only GROQ_API_KEY set	Groq

All API credentials are securely managed through server-side environment variables and are never exposed in browser-side code.

9. Installation and Running the Application

Repository URL: <https://github.com/nikshithareddyaela/AI-Chat-Assist>

Prerequisites:

- Node.js 18+
 - npm 9+
1. Clone the repository from GitHub.
 2. Navigate to the project directory.
 3. Install dependencies using `npm install`.
 4. Copy `.env.example` to `.env.local`.
 5. Configure environment variables in `.env.local`.
 6. Start the development server using `npm run dev`.

Command	Description
<code>npm run dev</code>	Start development server
<code>npm run build</code>	Create production build
<code>npm run start</code>	Run production build locally
<code>npm run lint</code>	Run ESLint
<code>npm run test</code>	Run unit tests
<code>npm run test:watch</code>	Run tests in watch mode during development

10. Deployment

The application is configured for deployment on Vercel. Environment variables configured locally should also be configured within the Vercel project environment settings before deployment.

Live Demo: <https://ai-chat-assist-mu.vercel.app>

11. Coding Standards and Best Practices

- Component-based architecture with clear separation of UI, hooks, utilities, and API logic.
- TypeScript-based contracts for messages, conversations, requests, responses, and streaming events.
- Secure server-side API handling with environment variable protection.
- Input validation on both client and server paths.
- Streaming response implementation with JSON fallback for better reliability.
- Sanitized Markdown rendering to reduce unsafe output risks.
- Reusable utility functions for storage, formatting, validation, and clipboard behavior.
- Unit tests for important utilities and components.

12. Future Enhancements

- User authentication support
- Conversation export functionality
- Request cancellation while streaming
- Enhanced accessibility support
- Cloud database synchronization
- Dark/light theme toggle

13. Conclusion

AI Chat Assist demonstrates strong modern frontend and full-stack fundamentals through scalable architecture, streaming AI integration, reusable components, secure API handling, responsive UI design, and robust state management. The implementation shows practical engineering judgment because the main requirements are mapped to maintainable code paths, validation, fallback behavior, and testable utilities.

Note: ChatHeader.tsx currently displays a static provider status label ('Ready · Groq API'). Provider resolution itself is dynamic in provider.ts, but the UI status text is not yet dynamically synchronized with the active provider.